

Plan de deploy — Besoiu Piese Auto pe VPS Linux

Magazin + admin pe server · unelte AI/scraper pe altă mașină · Apache · deploy prin git

Principiul de bază

Problema nu e aplicația, ci ce ai lângă ea. Din **~8.7 GB**, codul real e **~80 MB**; restul de ~8.6 GB sunt date regenerabile (`imports` , `tecdoc` , `feeds` , `backups`) și unelte de dev (Ollama, Python, Playwright, venv-uri) legate de Windows/Laragon.

Soluția: **un pachet de deploy curat de ~80 MB** care conține doar aplicația web. Datele grele și AI/scraper rămân pe mașina locală și trimit rezultatele pe server (mecanismul există deja: `SUPPLIER_SYNC_TOKEN` + agent local `supplier_sync`).

Unde e greutatea

Folder	Mărime	Ce e	Pe server?
<code>app/Backend/storage/imports</code>	4.2 GB	fișiere import furnizori	Nu (regenerabil)
<code>app/Backend/storage/tecdoc</code>	2.9 GB	cache TecDoc	Nu (regenerabil)
<code>app/Backend/storage/backups</code>	758 MB	backup-uri DB	Nu
<code>app/Backend/storage/ttc_image_library</code>	234 MB	imagini	Parțial
<code>app/Backend/storage/supplier_feeds</code>	227 MB	feed-uri	Nu (regenerabil)
<code>storage/scraper</code> + <code>tools/</code>	~825 MB	scraper Python + Playwright + venv	Nu (rulează local)
cod real (<code>admin</code> , <code>api</code> , <code>app/*</code> , <code>assets</code> , <code>modules</code>)	~80 MB	aplicația efectivă	Da

1. Ce EXCLUZI din pachet (nu urcă pe server)

- `app/Backend/storage/imports/**` , `.../tecdoc/**` , `.../backups/**` , `.../supplier_feeds/**` , `.../ttc_image_library/**` — regenerabile / se sincronizează separat
- `tools/` (scraper-core, stealth-browser-mcp) și `storage/scraper/` — rulează pe mașina locală
- `app/Import/**` (motor import/scraper Python, ~298 MB)
- toate `*/venv/` , `*/.venv/` , `node_modules/`
- `*.bat` (Windows/Laragon), `.git/` , `.cursor/` , `docs/` , fișierele `.json` / `.md` de audit din root
- `.env` reale (le creezi direct pe server, nu le urci)

2. Ce PĂSTREZI (aplicația reală)

`index.php` , `.htaccess` , `robots.php` , `sitemap.php` , `admin/` (fără `storage` + `vendor`) , `api/` , `assets/` , `modules/` , `app/` **fără** `Backend/storage` , `Import` , `Lib/scraper` — plus `app/Backend/vendor/` (generat pe server cu `composer install --no-dev`).

Foldere goale de recreat pe server (cu drept de scriere): `app/Storage/{cache,uploads,seo}` , `app/Backend/storage/{cache,logs,queue,async_jobs,rate_limits}` , `admin/storage/{cache,logs}` .

3. Cerințe server (VPS Linux)

- PHP 8.3** (min 8.1) cu extensiile: `pdo_mysql` , `mbstring` , `curl` , `zip` (pt. phpword), `openssl` (AES-256-GCM), `gd` , `intl` , `dom/xml` , `json`
- MySQL/MariaDB** — 2 baze: `besoiupieseauto.ro` (principală) + `caietcom_comenzilv` (legacy „Caiet de comenzi”)
- Apache** cu `mod_rewrite` + `mod_headers` + `mod_deflate` + `mod_expires` , `AllowOverride All`

- **Composer** (pt. build; se poate rula și local înainte de upload)
- DocumentRoot = rădăcina proiectului (unde e `index.php`)

4. Ce dezactivezi/repointezi în `.env` pe server

AI/scraper pe altă mașină:

- `OLLAMA_BASE_URL` → IP-ul mașinii locale (sau `OLLAMA_ENABLED=0` dacă serverul nu cheamă LLM)
- `STEALTH_BROWSER_ENABLED=0` (scraping doar local)
- `ASYNC_FILE_FALLBACK=1` (deja setat) → **Redis nu e obligatoriu** pe server
- `API_AUTOMATION_DISABLED=1`, `AI_AGENT_CYCLE_ENABLED=0` (fără cron AI automat)
- `APP_ENV=production`, `APP_URL=https://besoiupieseauto.ro`
- căi Windows (`F:/Poze`, `D:/laragon/...`) → căi Linux sau golite
- secrete reale: DB, `PAYMENT_WEBHOOK_KEY`, `PIESEAUTO_CREDENTIAL_KEY`, SmartBill/FanCourier/Sameday, `SUPPLIER_SYNC_TOKEN`

Trebuie configurate `.env` în 3-4 locuri: `app/Config/.env`, `app/Backend/.env`, `admin/.env`, (opțional `robot/.env`).

5. Pașii de deploy

- Build local: copiezi doar fișierele de la pct. 2 într-un folder `dist/` (via listă de excludere)
- `composer install --no-dev --optimize-autoloader` în `app/Backend/` (local sau pe server)
- Urci `dist/` pe server (git) în DocumentRoot
- Creezi cele 3-4 `.env` pe server și completezi secretele
- Import DB: creezi cele 2 baze + rulezi migrațiile din `admin/migrations/*.sql`
- Setezi permisiuni scriere pe folderele de storage (pct. 2)
- Configurezi cron minimal (pct. 6)
- Verifici: `/` (magazin), `/admin/` (login), `/api/...`

6. Cron minimal pe server (fără AI)

- `php admin/cron_cli/async_stale_job_sweeper.php` (la 60s) — curăță joburi blocate
- sync furnizori / dashboard **doar dacă** vrei automatizare pe server; altfel importurile vin de pe mașina locală prin `supplier_sync`

7. Atenție la deploy prin git

Important: la deploy prin git ajunge pe server **exact ce e comis în repo**. `.gitignore` -ul actual **nu** exclude tot ce e greu — lipsesc:

- `app/Backend/storage/tecdoc/` (2.9 GB)
- `app/Backend/storage/supplier_feeds/` (227 MB)
- `app/Backend/storage/ttc_image_library/` (234 MB)
- `tools/` și `storage/scraper/` (~825 MB)
- `app/Import/` (~298 MB)

Dacă acestea au ajuns deja în istoricul git, un simplu `git clone` le va trage oricum (chiar dacă le adaugi acum în `.gitignore`, rămân în istoric). Verifică cu `git ls-files` ce e efectiv versionat înainte de primul deploy — altfel „ușurarea” de la 8.7 GB la ~80 MB nu se întâmplă.

Pas următor opțional: scoaterea folderelor grele din repo și din istoric (comenzi `git rm --cached` + rescriere istoric), plus scripturi automate de build `dist/` .